Voxora

Team Workflow & Contribution Guide

A practical standard for folder ownership, branches, code quality, testing, security, and release

Purpose	Give every contributor a predictable place to work, a safe way to push code, and a shared definition of done.
Golden rule	Work on a feature branch, push that branch, and open a pull request into **develop**. Never push ordinary feature work directly to **main**.

Contents

1. Rules - 2. Structure - 3. Ownership - 4. Branches - 5. Code standards - 6. Contracts - 7. Testing - 8. Security - 9. Pull requests - 10. Release flow

1. Non-negotiable rules

- Never push directly to **main** or **develop**.

- Start every task from the latest **develop** branch.

- One task, one feature branch, one focused pull request.

- Put files only in the folder owned by the relevant module.

- Never commit passwords, API keys, service-account files, audio recordings, private transcripts, or .env files.

- Change shared contracts only through a focused pull request with updated mocks and tests.

- Every pull request must pass CI and receive at least one teammate review.

2. Standard repository structure

```
Voxora/
|-- apps/web/ React frontend
| `-- src/features/ conversation, audio, components, lib
|-- apps/api/ Fastify backend
| `-- src/ sessions, providers, security, config
|-- packages/contracts/ Shared TypeScript types and events
|-- infra/ AWS CDK infrastructure
|-- tests/e2e/ Playwright acceptance tests
|-- docs/contracts/ API, audio, event, and error contracts
`-- .github/workflows/ CI/CD workflows
```

Create new files inside the closest matching module. Do not put business logic in the repository root, and do not create competing folders for the same feature.

3. Member ownership

Member	Primary folders	Responsibility
1	apps/web/src/features/conversation, components	Pages, conversation layout, language selection, captions, responsive UI
2	apps/web/src/features/audio	Microphone permission, capture, PCM conversion, buffering, playback, mute
3	apps/api/src/providers	Provider interface, mock provider, Gemini adapter, provider errors
4	apps/api/src/modules/sessions	Session lifecycle, participants, preferences, validation
5	apps/api/src/security, config	Authentication, authorization, validation, CORS, rate limiting, secrets
6	infra, tests/e2e, .github, docs	AWS deployment, CI, end-to-end tests, release evidence

If a change crosses ownership boundaries, request review from every affected owner.

4. Branch workflow

The branch model is: **main** = stable releases, **develop** = shared integration, **feature/*** = individual tasks.

Create a branch

```
git fetch origin
git checkout develop
git pull origin develop
git checkout -b feature/member-1-conversation-screen
```

Push only your feature branch

```
git push -u origin feature/member-1-conversation-screen
```

Pull request targets

Feature branch -> **develop**
Release pull request: **develop** -> **main**

Recommended branch names

```
feature/member-1-conversation-ui
feature/member-2-audio-capture
feature/member-3-gemini-adapter
feature/member-4-session-api
feature/member-5-auth-security
feature/member-6-ci-e2e
fix/reconnect-after-network-loss
docs/update-team-guide
```

5. Commit and code standards

Use small, meaningful Conventional Commits:

```
feat(ui): add conversation language selector
fix(api): reject expired session codes
test(e2e): cover two-user conversation flow
docs(workflow): clarify branch rules
chore(ci): add typecheck job
```

Before committing:

```
pnpm typecheck
pnpm test
pnpm build
git diff --check
git status
```

Use TypeScript strict mode, explicit types at module boundaries, small functions, clear names, the existing formatter/linter, and tests for failure cases as well as successful flows.

6. Shared contracts

Shared contracts live in **packages/contracts/src/index.ts** and are documented in **docs/contracts/**.

- REST route names and request/response JSON.

- Session IDs, join codes, roles, expiry, and participant limits.

- Audio format: PCM16, 16 kHz, mono input; sequenced chunks; approximately 100 ms chunks.

- Translation events: partial transcript, final transcript, translated audio, status, and error.

- Stable error codes and safe user-facing messages.

Contract change process

- Open a focused contract pull request.

- Update packages/contracts.
- Update mock payloads and affected tests.
- Notify all affected owners before merging.

7. Testing requirements

- Session lifecycle: create, join, leave, end, duplicate join, expiry, invalid payloads.
- Audio: permission denied, missing device, mute, interruption, chunk ordering.
- Provider: disconnects, timeouts, unsupported languages, rate limits, invalid configuration.
- Security: unauthorized access, role violations, oversized requests, CORS, rate limiting.
- End-to-end: two browsers/devices, both directions, reconnect, network loss, and 10-20 minute sessions.

Test evidence records test ID, tester, date, browser/device, language pair, network condition, expected result, actual result, and pass/fail status.

8. Security and privacy

- Keep Gemini credentials on the backend only.
- Use AWS Secrets Manager in deployed environments.
- Commit only .env.example with placeholder values.
- Use HTTPS and WSS in staging and production.
- Never log raw audio, private transcripts, access tokens, or secret values.
- Do not store audio or transcripts by default.
- Validate all input and authorize every session action.
- A session join code must not grant host or administrative privileges.

9. Pull request checklist

- Linked GitHub Issue.
- Summary of the change and why it is needed.
- Module owner and affected folders.
- Test commands and results.
- Screenshots or logs where useful.
- No secrets or private data included.
- Shared contracts remain compatible.
- Documentation updated when required.

Do not merge when CI is failing, required reviewers have not approved, or a contract change has not been communicated.

10. Release flow

1	Feature branch passes local checks
2	Pull request targets develop
3	Reviewer checks scope, tests, security, and compatibility

4	CI passes; pull request merges into develop
5	Team runs the integrated staging smoke test
6	Team lead opens develop -> main release pull request
7	After approval, main is deployed and release notes are recorded

Quick command reference

```
pnpm install
pnpm dev
pnpm typecheck
pnpm test
pnpm build
git status
git add <your-files>
git commit -m "feat(scope): short description"
git push -u origin <your-feature-branch>
```

Never use **git push origin main** for ordinary feature work.